

13. ArenaX – Gaming Matchmaking and Player Management System

Course: Data Structures and Algorithms (DSA)

Name : Atharva Ashok Dalvi

Roll Number: 150096725179

Cohort: Sam Altman

Course : BTECH CSE (2025-29)

Problem Statement

Online multiplayer gaming platforms require efficient systems for managing player data, ranking players according to skill levels, maintaining matchmaking fairness, and selecting optimal servers with minimal latency.

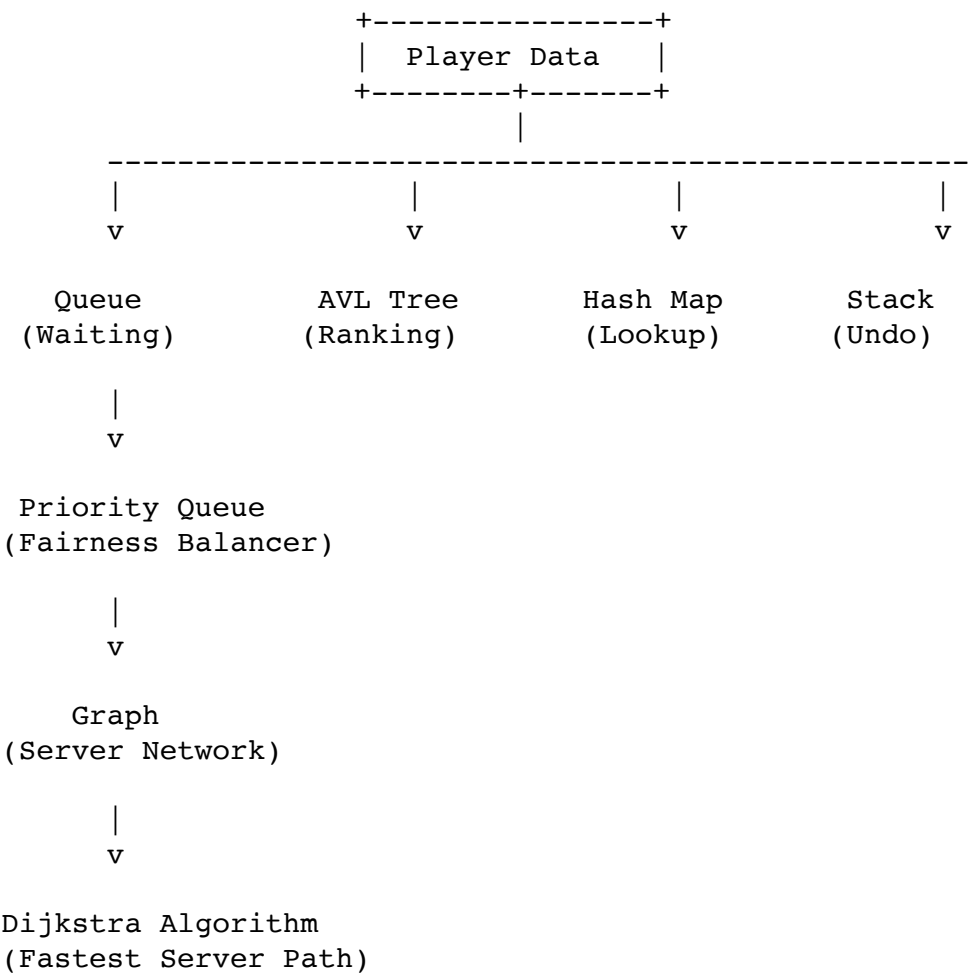
Traditional approaches may lead to slow player searches, unfair matchmaking, and inefficient server allocation. Therefore, a system is required that can efficiently organize players, prioritize waiting users, and optimize server connectivity using suitable data structures and algorithms.

Objectives

- 1. To efficiently store and manage player information.
- 2. To provide fast player lookup using unique IDs.
- 3. To maintain player rankings based on skill levels.
- 4. To implement matchmaking queues following arrival order.
- 5. To support undo operations for player management.
- 6. To prioritize players with longer waiting times.
- 7. To optimize server communication using shortest path algorithms.
- 8. To demonstrate the real-world application of advanced data structures.

Design / Architecture

System Architecture



Module Description

Queue

- Maintains players in matchmaking order.

Stack

- Stores recently added players for undo operations.

Hash Map

- Provides quick player search using IDs.

AVL Tree

- Maintains balanced player rankings.

Priority Queue

- Prioritizes players with maximum waiting time.

Graph

- Represents connections between game servers.

Dijkstra Algorithm

- Calculates shortest latency paths.

Algorithm Description

Algorithm 1: Add Player

1. Create player object.
2. Insert player into AVL Tree.
3. Add player to waiting queue.
4. Push player onto stack.
5. Store player in hash map.
6. Insert waiting time into priority queue.

Algorithm 2: Player Lookup

1. Enter Player ID.
2. Search ID in Hash Map.
3. If found, display details.
4. Otherwise display "Player Not Found".

Algorithm 3: AVL Ranking

1. Insert player based on skill value.
2. Calculate balance factor.
3. Perform rotations if tree becomes unbalanced.
4. Display players using inorder traversal.

Algorithm 4: Fairness Balancer

1. Store players in max heap using waiting time.
2. Extract player with maximum waiting time.
3. Match longest waiting players first.

Algorithm 5: Dijkstra's Algorithm

1. Initialize source distance as 0.
2. Set all other distances to infinity.
3. Use priority queue to process nearest node.
4. Update neighboring distances.
5. Repeat until all nodes are processed.
6. Display shortest latency paths.

Complexity Analysis

Data Structure / Algorithm	Time Complexity	Space Complexity
Queue Push/Pop	O(1)	O(n)
Stack Push/Pop	O(1)	O(n)
Hash Map Lookup	O(1) Average	O(n)
AVL Tree Insert	O(log n)	O(n)
AVL Tree Search	O(log n)	O(n)
Sorting Players	O(n log n)	O(1)
Priority Queue Insert	O(log n)	O(n)
Priority Queue Delete	O(log n)	O(n)
Dijkstra Algorithm	O((V+E) log V)	O(V)

Overall Complexity

The overall system performance is primarily influenced by:

- AVL Tree Operations
- Player Sorting
- Dijkstra Algorithm

Overall Complexity: O(n log n)

Screenshots of Execution

```

...
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
▼ TERMINAL

atharva@Atharvas-MacBook-Air-5 dsaproject % ./s
2. WAITING QUEUE (arrival order)
=====
ID:1 Alice Skill:85
ID:2 Bob Skill:42
ID:3 Charlie Skill:73
ID:4 Diana Skill:91
ID:5 Eve Skill:58
ID:6 Frank Skill:66
=====

3. UNDO LAST ADDITION (stack)
=====
Player 'Frank' re-queued via undo.
=====

4. PLAYER LOOKUP BY ID (hash map)
=====
Found ID 2 -> Bob Skill:42
Found ID 4 -> Diana Skill:91
ID 99 -> Not found.
=====

5. PLAYERS SORTED BY SKILL (descending)
=====
Diana Skill:91
Alice Skill:85
Charlie Skill:73
Frank Skill:66
Eve Skill:58
Bob Skill:42

```

```

=====
6. AVL RANK INDEX (ascending inorder)
=====
ID:2 Name:Bob Skill:42
ID:5 Name:Eve Skill:58
ID:6 Name:Frank Skill:66
ID:3 Name:Charlie Skill:73
ID:1 Name:Alice Skill:85
ID:4 Name:Diana Skill:91
=====

7. FASTEST SERVER PATH (Dijkstra)
=====
--- Minimum Latency from Server 0 ---
Server 0 : 0 ms
Server 1 : 20 ms
Server 2 : 10 ms
Server 3 : 15 ms
Server 4 : 27 ms
=====

8. FAIRNESS BALANCER (max-heap by wait time)
=====
Priority order (longest wait first):
#1 ID:2 Bob WaitTime:60s
#2 ID:6 Frank WaitTime:50s
#3 ID:4 Diana WaitTime:45s
#4 ID:1 Alice WaitTime:30s
#5 ID:5 Eve WaitTime:20s
#6 ID:3 Charlie WaitTime:15s
ArenaX simulation complete.
o atharva@Atharvas-MacBook-Air-5 dsaproject %

```

Results and Findings

Results

1. Queue successfully maintained player arrival order.
2. Stack provided efficient undo functionality.
3. Hash Map enabled constant-time player lookup.
4. AVL Tree maintained balanced ranking structure.
5. Priority Queue ensured fairness in matchmaking.
6. Dijkstra Algorithm correctly computed shortest server paths.

Findings

- Combining multiple data structures improves system efficiency.
- AVL Trees outperform normal BSTs in worst-case scenarios.
- Hash Maps are highly effective for frequent search operations.
- Priority Queues help reduce player waiting dissatisfaction.
- Graph algorithms are useful for network optimization.

Conclusion

The ArenaX Gaming Matchmaking and Player Management System successfully demonstrates the practical use of various data structures and algorithms in solving real-world gaming problems.

The project integrates Queue, Stack, Hash Map, AVL Tree, Priority Queue, Graphs, and Dijkstra's Algorithm into a single system capable of handling player management, ranking, matchmaking fairness, and server optimization efficiently.

The implementation highlights the importance of selecting appropriate data structures to achieve high performance and scalability in software applications.

GitHub Repository Link

GitHub Repository:

<https://github.com/AtharvaD2007/Digital-Game-Matchmaker-Pipeline/tree/main>